

UNIT III

3

GUI Programming

Syllabus

Designing Graphical User Interfaces in Java, Components and Containers, Basics of Components, Using Containers, Layout Managers, AWT Components, Adding a Menu to Window, Extending GUI Features Using Swing Components, Java Utilities (java.util Package) The Collection Framework : Collections of Objects, Collection Types, Sets, Sequence, Map, Understanding Hashing, and Use of Array List & Vector.

Contents

- 3.1 *Designing Graphical User Interfaces in Java*
- 3.2 *Components and Containers*
- 3.3 *Basics of Components*
- 3.4 *Extending GUI Features Using Swing Components*
- 3.5 *Java Utilities (java.util Package)*
- 3.6 *The Collection Framework*
- 3.7 *Collections of Objects and Types*
- 3.8 *List Interface*
- 3.9 *Vector*
- 3.10 *Set Interface*
- 3.11 *Map Interface*
- 3.12 *Multiple Choice Questions*

3.1 Designing Graphical User Interfaces in Java

- Swing is another approach of graphical programming in Java.
- Swing creates highly interactive GUI applications.
- It is the most flexible and robust approach.

3.1.1 Difference between AWT and Swing

Sr. No.	AWT	Swing
1.	The Abstract Window ToolKit is a heavy weight component because every graphical unit will invoke the native methods.	The Swing is a light weight component because it's the responsibility of JVM to invoke the native methods.
2.	The look and feel of AWT depends upon platform.	As Swing is based on Model View Controller pattern, the look and feel of swing components is independent of hardware and the operating system.
3.	AWT occupies more memory space.	Swing occupies less memory space.
4.	AWT is less powerful than Swing.	Swing is extension to AWT and many drawbacks of AWT are removed in Swing.

3.2 Components and Containers

The most commonly used component classes are -

Component	Purpose
AbstractButton	Abstract class for the buttons
ButtonGroup	It creates the group of buttons so that they behave in mutually exclusive manner.
JApplet	The swing applet class
JButton	The swing button class
JCheckBox	The swing check box
JComboBox	The swing combo box
JLabel	The swing label component

JRadioButton	The radio button component
JTextArea	The text area component
TextField	The text field component
JSlider	The slider component

We will learn and understand these components with the help of programming examples.

3.3 Basics of Components

In order to display any JComponent on the GUI, it is necessary to add this component to the container first. If you do not add these components to container then it will not be displayed on the GUI. The swing class component hierarchy is as shown by following Fig. 3.3.1. (See Fig. 3.3.1 on next page).

There are two important features of swing components - Firstly, all the component classes begin with the letter J and secondly all these GUI components are descendant of JComponent class. Let us now learn how to place these components on the GUI.

Review Question

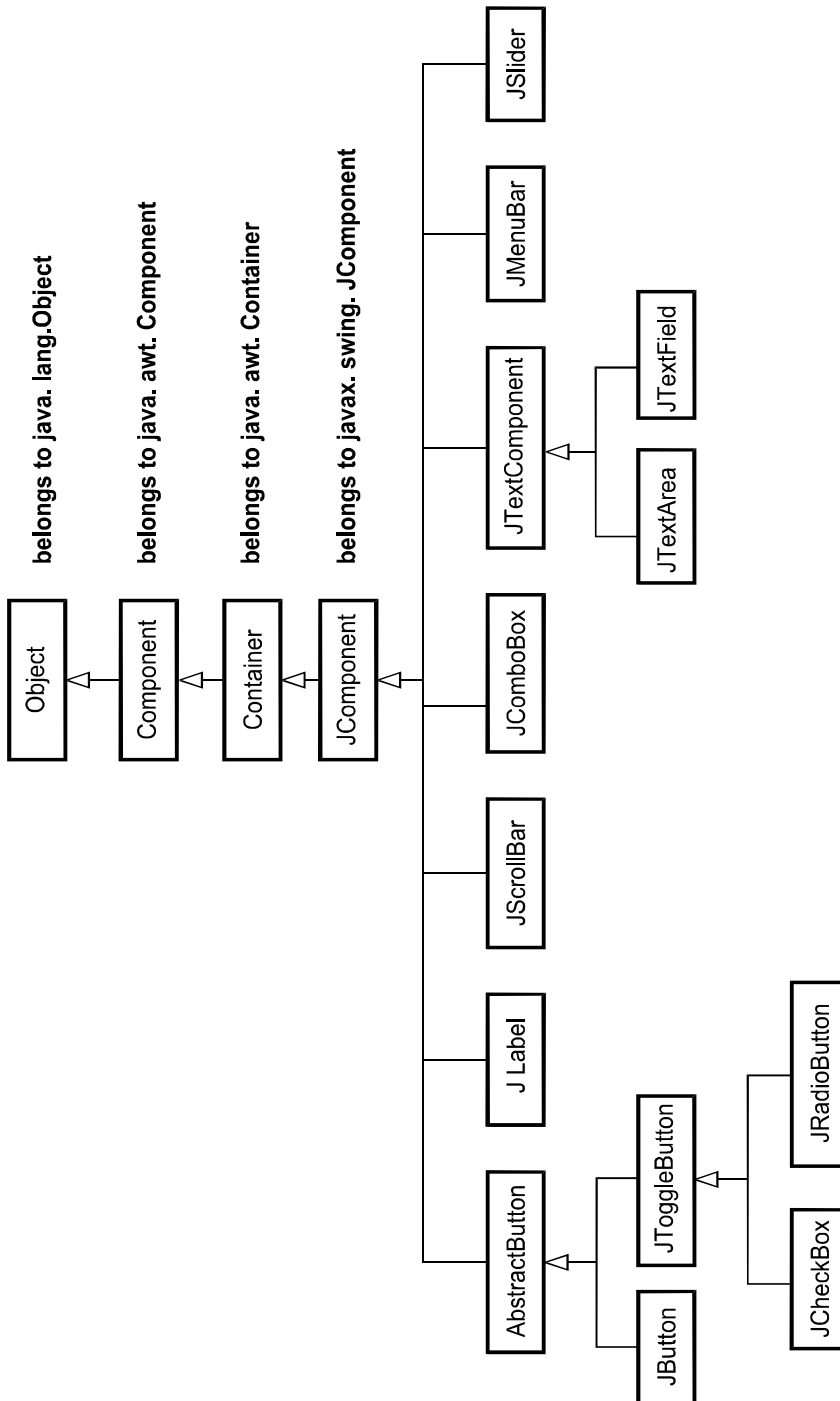
1. List and briefly discuss the swing components in Java.

3.4 Extending GUI Features Using Swing Components

- Swing is a large system. It is one part of Java Foundation Classes (JFC).
- It has got good **look** and **feel**.
- There is a rich collection of swing components that can be used to provide the advanced user interface.
- Various classes that can be used for placing the components are JApplet, JLabel, JTextField, JButton, JCheckBox, JTree, JTabbedPane and so on.

3.4.1 JApplet

The **JApplet** is a fundamental swing class. It extends the Applet class. This is much more powerful than the Applet class. It supports the **pane**. Various panes are content pane, glass pane and root pane. The **add** method can be used to add the **content pane**. The add method of **Container** class can be used to add the components on the GUI.

**Fig. 3.3.1 Swing class component hierarchy**

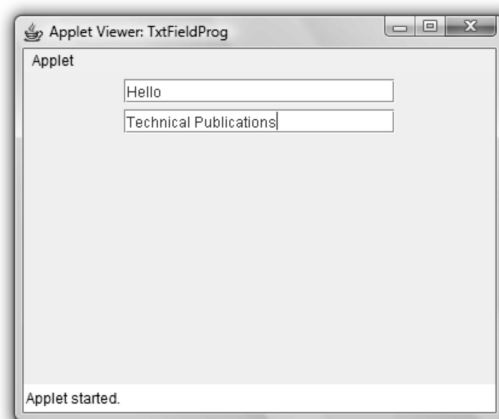
Example program

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="TxtFieldProg" width=300 height=200>
</applet>
*/
public class TxtFieldProg extends JApplet
{
    public void init()
    {
        JTextField T;
        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());
        T=new JTextField("Hello",20);
        contentPane.add(T);
        T=new JTextField(20);
        contentPane.add(T);
    }
}
```

For getting the output of the above program following commands can be given on the command prompt -

```
F:\SwingProg>javac TxtFieldProg.java
F:\SwingProg>AppletViewer TxtFieldProg.java
```

The Applet viewer will display the GUI as follows -

Output

Program explanation

To place the control textfield on the GUI we have followed following steps -

1. Using the **getContentPane** method an object for the **Container** class is created. This object is taken in the variable *contentPane*.

2. The Layout is set using the *contentPane* object by the statement

```
contentPane.setLayout(new FlowLayout());
```

The default layout is **FlowLayout** but you can set other layout managers such as **GridLayout**, **BorderLayout** and so on.

3. After setting the layout manager, the **TextField** component can be created and placed using **add** method. In above program, we have created two text fields. In the first text field, the string "Hello" is already written during its creation but the second text field is kept blank and some string can be written into it during the execution.

3.4.2 Creating Frames

- A Frame is a top-level window with a title and a border. The Frames in java works like the main window where the components like JButton, JLabel, JComboBox and so on can be placed. In Java swing the top-level windows are represented by the **JFrame** class. For creating the frame following statement can be used -

```
JFrame f=new JFrame("My First Frame Program");
```



- The frames are not visible initially, hence we have to make them visible by `f.setVisible(true);`
- The close button of the frame by default performs the hide operation for the JFrame. Hence we have to change this behavior to window close operation by setting the `setDefaultCloseOperation()` to **EXIT_ON_CLOSE** value.
- Any suitable size of the frame can be set by **setSize(int height,int width)** function.

Example 3.4.1 Write a JFrame with a hello world program.

Solution :

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
```

```
public class FrameProg extends JFrame
{
    public static void main(String[] args)
    {
        new FrameProg();
    }
    FrameProg()
    {
        JFrame f=new JFrame("Frame Demo");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
        f.setSize(300,300);
        Container content=f.getContentPane();
        content.setLayout(new FlowLayout());
        content.add(new JLabel("Hello World"));

    }
}
```

Output

3.4.3 Label and ImageIcon

- The icons are encapsulated by **ImageIcon** class. The constructors are -

```
ImageIcon(string fname)
ImageIcon(URL url)
```

The *fname* denotes the name of the file which contains the icon. The *url* denotes the resource of image.

- The **JLabel** is a component for placing the label component. The **JLabel** is a subclass of **JComponent** class. The syntax for the JLabel is -

```
JLabel(Icon ic);
Label(String s);
JLabel(String s, Icon ic, int alignment)
```

- The alignment can be LEFT, RIGHT, CENTER, LEADING and TRADING. The icons and text methods associated with the label are -

```
Icon getIcon()
void setIcon(Icon ic)    ← ic represents the icon file
String getText();
void setText(String s); ← s represents the string
```

- Following simple Java Program illustrates the use of JLabel component -

Java Program[LabelProg.java]

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="LabelProg" width=300 height=200>
</applet>
*/
public class LabelProg extends JApplet
{
    public void init()
    {
        Container contentPane=getContentPane();
        ImageIcon i=new ImageIcon("innocence.gif");
        JLabel L1=new JLabel("Innocence",i,JLabel.LEFT);
        contentPane.add(L1);
    }
}
```


Output**3.4.4 TextField**

- The **TextField** is extended from the **JComponent** class. The **TextField** allows us to add a single line text.
- The syntax of using **TextField** is -

```
TextField();  
TextField(int col_val);  
TextField(String s,int col_val);  
TextField(String s);
```

- The program making use of TextField is as given below -

Java Program[TxtFieldProg.java]

```
import java.awt.*;  
import javax.swing.*;  
/*  
<applet code="TxtFieldProg" width=300 height=200>  
</applet>  
*/  
public class TxtFieldProg extends JApplet  
{  
    public void init()  
    {  
        JTextField T;  
        Container contentPane=getContentPane();  
        contentPane.setLayout(new FlowLayout());
```

```
T=new JTextField("Hello",20);
contentPane.add(T);
T=new JTextField(20);
contentPane.add(T);
}
}
```

- For getting the output of the above program following commands can be given on the command prompt -

```
F:\SwingProg>javac TxtFieldProg.java
```

```
F:\SwingProg>AppletViewer TxtFieldProg.java
```

The Applet viewer will display the GUI as follows -

Output



Program explanation

To place the control textfield on the GUI we have followed following steps -

1. Using the `getContentPane` method an object for the container class is created. This object is taken in the variable `contentPane`.
2. The layout is set using the `contentPane` object by the statement

```
contentPane.setLayout(new FlowLayout());
```

The default layout is `FlowLayout` but you can set other layout managers such as `GridLayout`, `BorderLayout` and so on.

3. After setting the layout manager, the `TextField` component can be created and placed using `add` method. In above program, we have created two text fields. In the first text field, the string "Hello" is already written during its creation but the second text field is kept blank and some string can be written into it during the execution.

3.4.5 TextArea

The `JTextArea` is a GUI control which allows us to write multi-line text. We can set the desired number of rows and number of columns so that a long big message can be written using this control.

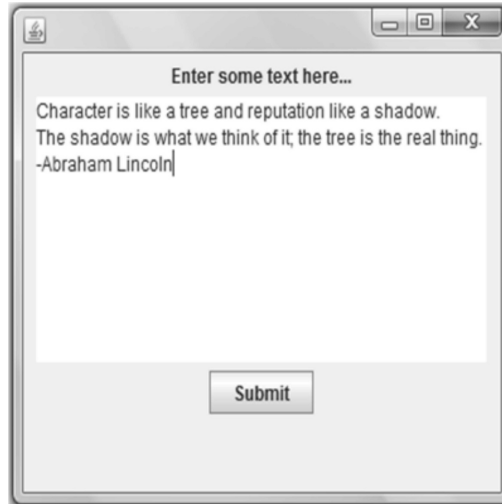
The syntax for JTextArea is

```
JTextArea(String str,int rows,int cols);
```

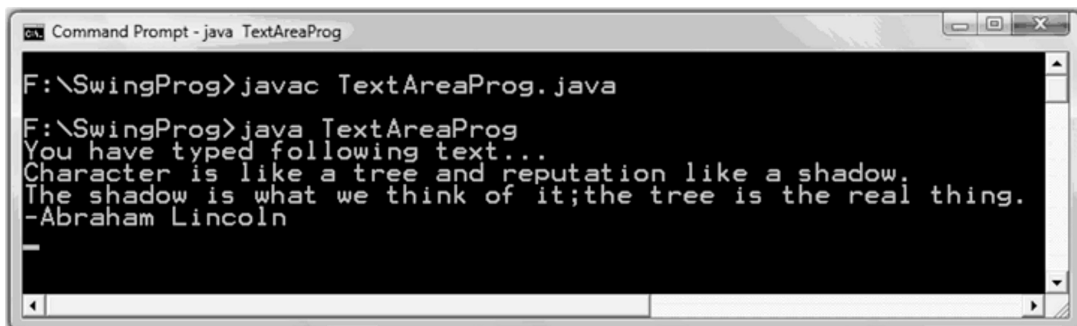
In the following Java program, there are two GUI controls on the window - the textarea and the push button. The idea is that : just type something within the text area and then click the push button. Whatever text you type will get displayed on the console window.

Java Program[TextAreaProg.java]

```
import java.awt.event.*;
import java.awt.*;
import javax.swing.*;
class TextAreaProg
implements ActionListener
{
    JTextArea TA;
    public static void main(String[] args)
    {
        new TextAreaProg(); //invoke the function for demonstrating TextArea control
    }
    TextAreaProg() //definition of the function
    {
        JFrame f=new JFrame(); //create a Frame window
        Container content=f.getContentPane();//get the Container class object
        content.setLayout(new FlowLayout());//set the layout manager
        JLabel L=new JLabel("Enter some text here...");//create a Label control
        content.add(L); //using add method add the label on the GUI
        TA=new JTextArea("",10,20); //create a TextArea control with rows=10 and col=20
        content.add(TA);//using add method add the TextArea control on GUI
        JButton B=new JButton("Submit");//create a push button control
        content.add(B);//place this control on the GUI using add method
        B.addActionListener(this);//invoke action listener for button click event
        f.setSize(300,300);//set the size of the frame window
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);//exit on closing
        f.setVisible(true);//set the visibility of frame as true
    }
    public void actionPerformed(ActionEvent e)//handling the event when button is clicked
    {
        String s=TA.getText();//get the string typed in textarea
        System.out.println("You have typed following text...\n"+s);//display it on the consol
    }
}
```

Output

Now if we click **Submit** button then on the command-prompt window we can see following text-

**3.4.6 Buttons**

- The swing push button is denoted by using **JButton** class.
- The swing button class provides the facilities that can not be provided by the applet button class. For instance you can associate some image file with the button.
- The swing button classes are subclasses of **AbstractButton** class.
- The **AbstractButton** class generates action events when they are pressed. These events can be associated with the Push buttons.
- We can associate an icon and/or string with the **JButton** class. The syntax of JButton is

```
JButton(Icon ic);  
JButton(String s);  
JButton(String s,Icon ic);
```

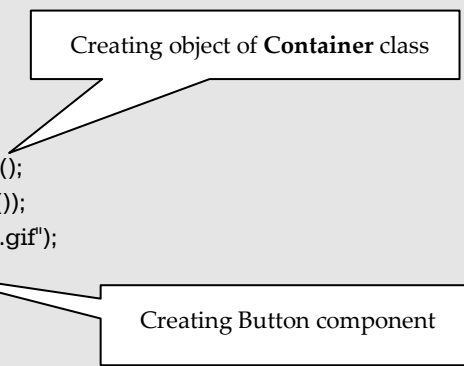
- Following program illustrates the use of Icon and the string for the JButton class -

Java Program[ButtonProg.java]

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="ButtonProg" width=500 height=300>
</applet>
*/
public class ButtonProg extends JApplet//inherited from JApplet
implements ActionListener
{
    JTextField T;
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());
        ImageIcon apple=new ImageIcon("apple.gif");
        JButton B1=new JButton(apple);
        B1.setActionCommand("Apple");
        B1.addActionListener(this);
        contentPane.add(B1);//adding button on GUI

        ImageIcon orange=new ImageIcon("orange.gif"); //Creating image icon
        JButton B2=new JButton(orange); //associating image with button
        B2.setActionCommand("Orange");
        B2.addActionListener(this); //button pressed event
        contentPane.add(B2);

        ImageIcon grapes=new ImageIcon("grapes.gif");
        JButton B3=new JButton(grapes);
        B3.setActionCommand("Grapes");
        B3.addActionListener(this);
        contentPane.add(B3);
        T=new JTextField(20);
        contentPane.add(T);//placing the textfield on the GUI
    }
    public void actionPerformed(ActionEvent e)
    {
        T.setText(e.getActionCommand()); //retrieving the text associated with button
    }
}
```

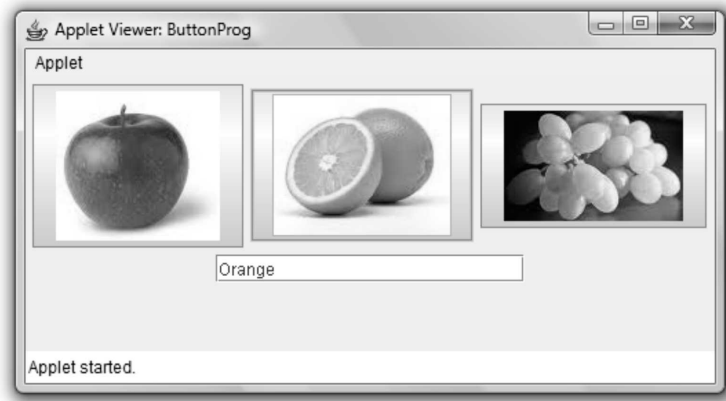


For getting the output open the **command-prompt** and give the following commands -

```
F:\SwingProg>javac ButtonProg.java  
F:\SwingProg>AppletViewer ButtonProg.java
```

and you will get the applet as follows -

Output



Program explanation

The above program is inherited from the **JApplet** class. In the init method, we have written the code -

1. Create a container object by using the **getContentPane** method. This object is now in the variable **contentPane**.
2. A layout manager such as **FlowLayout** is used to set the layout of the GUI.
3. The button components are then placed on the GUI using the **add** method.
4. These push buttons are associated with some images using **ImageIcon** class. To this **ImageIcon** class appropriate gif file is passed as an argument. Thus corresponding image gets associated with each corresponding push button.
5. We have associated an **ActionListener** event with this program. This is necessary to handle the events when a push button gets pressed. We have associated some command string when the particular button is pressed. This can be done using **setActionCommand** method. The event handler can be invoked using the methods **addActionListener(this)**. When this a call to this method is given the control goes to the function **actionPerformed**. In this method we are simply displaying the appropriate command string in the textbox.

3.4.7 Checkboxes

- The Check Box is also implementation of **AbstractButton** class. But the immediate superclass of **JCheckBox** is **JToggleButton**.
- The JCheckBox supports two states true or false.
- We can associate an icon, string or the state with the checkboxes. The syntax for the Check Box will be -

```
JCheckBox(Icon ic);
JCheckBox(Icon ic, boolean state);
JCheckBox(String s);
JCheckBox(String s,boolean state);
JCheckBox(String s,Icon ic,boolean state);
```

- Following program illustrates the use of check box -

Java Program[CheckBoxProg.java]

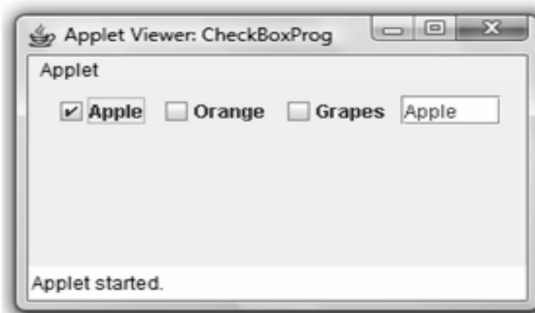
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="CheckBoxProg" width=300 height=300>
</applet>
*/
public class CheckBoxProg extends JApplet
implements ItemListener
{
    JTextField T;
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());
        JCheckBox chk1=new JCheckBox("Apple");
        chk1.addItemListener(this);// invoking the event handler
        contentPane.add(chk1);
        JCheckBox chk2=new JCheckBox("Orange");
        chk2.addItemListener(this); // invoking the event handler
        contentPane.add(chk2);
        JCheckBox chk3=new JCheckBox("Grapes");
        chk3.addItemListener(this); // invoking the event handler
        contentPane.add(chk3);
        T=new JTextField(5);
        contentPane.add(T);
        ButtonGroup bg=new ButtonGroup();
```

```
bg.add(chk1);
bg.add(chk2);
bg.add(chk3);
```

ButtonGroup helps the objects to behave mutually exclusive.

```
}
public void itemStateChanged(ItemEvent e)
{
    JCheckBox chk=(JCheckBox)e.getItem();
    T.setText(chk.getText());//displaying the appropriate string in textbox
}
}
```

Output



Program explanation

In above program, the **ItemListener** event is implemented. Using the **addItemListener(this)** method the event handler function **itemStateChanged** is invoked. Thus on the clicking corresponding checkbox, the appropriate string will be displayed in the text field.

3.4.8 Radio Buttons

- The **JRadioButton** is a subclass of **JToggleButton**. This control is similar to the checkboxes.
- The syntax for the Radio Box will be -

```
JRadioButton(Icon ic);
JRadioButton (Icon ic, boolean state);
JRadioButton (String s);
JRadioButton (String s,boolean state);
JRadioButton (String s,Icon ic,boolean state);
```

- Following program illustrates the use of radio buttons.

Java Program[RadioButProg.java]

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="RadioButProg" width=300 height=300>
</applet>
*/
public class RadioButProg extends JApplet
implements ActionListener
{
    JTextField T;
    public void init()
    {

        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());

        JRadioButton rb1=new JRadioButton("Apple");
        rb1.addActionListener(this);
        contentPane.add(rb1);

        JRadioButton rb2=new JRadioButton("Orange");
        rb2.addActionListener(this);
        contentPane.add(rb2);
        JRadioButton rb3=new JRadioButton("Grapes");
        rb3.addActionListener(this);

        contentPane.add(rb3);
        T=new JTextField(5);
        contentPane.add(T);
        ButtonGroup bg=new ButtonGroup();
        bg.add(rb1);
        bg.add(rb2);
        bg.add(rb3);
    }
    public void actionPerformed(ActionEvent e)
    {

        T.setText(e.getActionCommand());
    }
}
```

Output**Program explanation**

We have used **ActionListener** event to handle the radio button click. The **addActionListener** method is invoked when the button gets clicked. The appropriate command string can be displayed in the textfield.

3.4.9 Lists

JList is a component that displays list of text items. User can select one or multiple items from this list.

Various components of JList component are

JList()	Creates a JList with an empty, read-only, model.
JList(ary[] listData)	Creates a JList that displays the elements in the specified array.
JList(ListModel<ary> dataModel)	Creates a JList that displays elements from the specified, non-null, model.

Various methods of this component are -

Method	Description
int getSelectedIndex()	It returns the index of selected item of the list.
void setListData(Object[] list)	It is used to create a read-only ListModel from an array of objects.

Java Program

```
import javax.swing.*;
import java.awt.*;
/*
<applet code="ListDemo" width=200 height=200>
</applet>
*/
public class ListDemo extends JApplet
{
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new BorderLayout());
        String [] str={"Windows98\n","Windows2000\n","Windows7\n","Windows8\n","Windows10"};
        JList list = new JList(str);
        list.setBounds(100,100, 75,75);
        contentPane.add(list);
    }
}
```

Output



3.4.10 Choices

- **JList** and **JCombobox** are very similar components but the **JList** allows to select multiple selections whereas the **JCombobox** allows only one selection at a time.
- A combo box is a combination of text field and the drop down list. The **JComboBox** is a subclass of **JComponent** class.

Java Program[ComboBoxProg.java]

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="ComboBoxProg" width=300 height=300>
</applet>
*/
public class ComboBoxProg extends JApplet
implements ItemListener
{
    JLabel L;
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());
        JComboBox co=new JComboBox();
        co.addItem("apple");
        co.addItem("orange");
        co.addItem("grapes");
        co.addItemListener(this);
        contentPane.add(co);

        L=new JLabel(new ImageIcon("apple.gif"));
        contentPane.add(L);

    }
    public void itemStateChanged(ItemEvent e)
    {
        String str=(String)e.getItem();
        L.setIcon(new ImageIcon(str+".gif")); //invokes the appropriate image file
    }
}
```

Output**Program explanation**

Using the **new JComboBox()** the object for the combo box can be created. This object then invokes the method **addItem** for adding the items in the combo box. In the event handler function **itemStateChanged** the **getItem** method is used to retrieve the selected string.

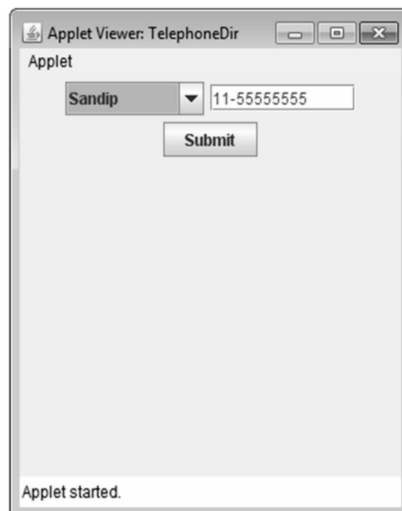
Example 3.4.2 *Create a phonebook look-up application that displays the phone number for the person selected and looks up the name for an entered phone number. Create a GUI that consists of a pull-down list of names, a text field for entry of phone numbers to look up and a command button to activate the look-up operation. It is your job to add the event handling to the application. There are three different events that must be handled. When the user changes the name in the pull-down list, the application should look up the appropriate telephone number in the phonebook. When the user enters a phone number in the text field and presses return or selects the button, the application should look up the number in the reverse phonebook and change the name in the pull-down list. Finally, the application should restrict input in the text field to digits and the '-' character. If a look-up operation fails, the application should simply beep.*

Solution :

```
//Program for handling the telephone Directory using event handling
import java.util.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/*
<applet code="TelephoneDir" width=300 height=300>
</applet>
*/
public class TelephoneDir extends JApplet
implements ItemListener, ActionListener
{
    JTextField T;
    JComboBox co;
    JButton B;
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new FlowLayout());
        co=new JComboBox();
        co.addItem("Select an Item");
        co.addItem("Archana");
        co.addItem("Supriya");
        co.addItem("Jayashree");
        co.addItem("Shivraj");
        co.addItem("Sandip");
        contentPane.add(co);
        co.addItemListener(this);
        T=new JTextField(10);
        contentPane.add(T);
        T.addActionListener(this);
        B=new JButton("Submit");
        contentPane.add(B);
        B.addActionListener(this);
    }
    public void itemStateChanged(ItemEvent e)
    {
        String str=(String)e.getItem();
        if(str=="Archana")
            T.setText("11-11111111");
        else if(str=="Supriya")
            T.setText("11-22222222");
        else if(str=="Jayashree")
```

```
T.setText("11-33333333");
else if(str=="Shivraj")
T.setText("11-44444444");
else if(str=="Sandip")
T.setText("11-55555555");
else
{
if((str=="") || (T.getText()==""))//generating Beep
System.out.println("\007");
}
}
public void actionPerformed(ActionEvent e)
{
String str=(String)T.getText();
co.setSelectedItem(str);
if(co.getSelectedItem()=="Select an Item");//no item selected
System.out.println("\007\007");//generating Beep
}
}
```

Output



Example 3.4.3 Write a program to create product enquiry form using frames.

Solution :

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class FrameProg extends JFrame
{
```

```
public static void main(String[] args)
{
    new FrameProg();
}
FrameProg()
{
    JFrame f=new JFrame("Enquiry Form");
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
    f.setSize(800,300);
    Container content=f.getContentPane();
    content.setLayout(new FlowLayout(FlowLayout.CENTER));
    content.add(new JLabel("Name:"));
    content.add(new JTextField(10));
    content.add(new JLabel("Address:"));
    content.add(new JTextField(20));
    content.add(new JLabel("Product:"));
    JComboBox co=new JComboBox();
    co.addItem("Mobile Phones");
    co.addItem("Laptops");
    co.addItem("ipods");
    co.addItem("Tablet PC");
    content.add(co);
    JButton b=new JButton("Submit");
    content.add(b);
}
}
```

Example 3.4.4 *How will you display an image on the frame in a window using java ?*

Solution :

```
import javax.swing.*;
import java.awt.*;
public class ImgFrameDemo extends JFrame
{
    public void display()
    {

        JFrame frame = new JFrame("Displaying Image On Frame");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 400);
        frame.setResizable(false);
        frame.setLocationRelativeTo(null);
        // Inserts the image icon
    }
}
```



```
ImageIcon image = new ImageIcon("img1.jpg");
JLabel label1 = new JLabel(" ", image, JLabel.CENTER);
frame.getContentPane().add(label1);
frame.validate();
frame.setVisible(true);
}
public static void main(String[] args)
{
    ImgFrameDemo obj = new ImgFrameDemo();
    obj.display();
}
}
```

Output



3.4.11 ScrollPane

- The ScrollPane is a rectangular area in which some component can be placed.
- The component can be viewed with the help of **horizontal** and **vertical** scroll bars.
- Using the **JScrollPane** class the component can be added in the program.
- The **JScrollPane** class extends the **JComponent** class.
- There are **three constructors** that can be used for this component -

```
JScrollPane(Component component)
JScrollPane(int vscrollbar, int hscrollbar)
JScrollPane(Component component, int vscrollbar, int hscrollbar)
```

The *component* represents the reference to the component. The *vscrollbar* and *hscrollbar* are the integer values for the vertical and horizontal scroll bars. These values can be defined by the constants such as

Constant	Meaning
HORIZONTAL_SCROLLBAR_ALWAYS	It always displays the horizontal scroll bar.
HORIZONTAL_SCROLLBAR_NEEDED	It displays the horizontal scrollbar if required.
VERTICAL_SCROLLBAR_ALWAYS	It always displays the vertical scroll bar.
VERTICAL_SCROLLBAR_NEEDED	It displays the vertical scrollbar if required.

- Following is a simple program which illustrates the use of scrollpane.

Step 1 : Create a label.

Step 2 : Create a panel.

Step 3 : Use an image with the help of label.

Step 4 : Add the label on the panel.

Step 5 : Create a content pane.

Step 6 : Create a scrollpane component by passing the image (within a panel) as a *component* to it.

Step 7 : Add the the scrollpane component to the content pane component.

Java Program

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="ScrollPaneDemo" width=150 height=150>
</applet>
*/
public class ScrollPaneDemo extends JApplet
{
    public void init()
    {
        Container contentPane = getContentPane();
        contentPane.setLayout(new BorderLayout());
        JPanel mypanel = new JPanel();
        JLabel L1 = new JLabel();
        ImageIcon i = new ImageIcon("img.jpg");
        L1.setLocation(20, 100);
        L1.setSize(120, 120);
        L1.setIcon(i);
        mypanel.add(L1);
    }
}
```

```

int vscrollbar = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int hscrollbar = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp = new JScrollPane(mypanel, vscrollbar, hscrollbar);
contentPane.add(jsp, BorderLayout.CENTER);
}
}

```

Output



Example 3.4.5 *List does not support scrolling. Why ? How this can be remedied ? Explain with an example.*

Solution : List is a component which simply displays the list of items. But this component does not support scrolling. The scrolling facility can be added to this component by using the scroll pane. Following are the steps that can be carried out for that purpose -

Step 1 : Create the content pane.

Step 2 : Create a list of items. (Preferably list of lots of items)

Step 3 : Create a scroll pane component by setting the values to vertical and horizontal component `ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED` and `ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED`;

Step 4 : Then pass the JList component to scroll pane component.

Step 5 : Then add this scroll pane component to content pane.

Following program illustrates this idea -

Java Program

```

import javax.swing.*;
import java.awt.*;
import javax.swing.tree.*;

```

```

/*
<applet code="ListDemo" width=150 height=90>
</applet>
*/

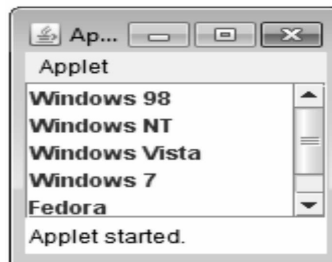
public class ListDemo extends JApplet
{
    public void init()
    {
        Container contentPane = getContentPane();

        contentPane.setLayout(new BorderLayout());
        String[] str = { "Windows 98\n", "Windows NT\n", "Windows Vista\n", "Windows
7\n", "Fedora\n", "Ubuntu" };
        JList list = new JList(str);

        int vscrollbar = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
        int hscrollbar = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;

        JScrollPane mypane = new JScrollPane(list, vscrollbar, hscrollbar);
        contentPane.add(mypane, BorderLayout.CENTER);
    }
}

```

Output

3.4.12 Scrollbar

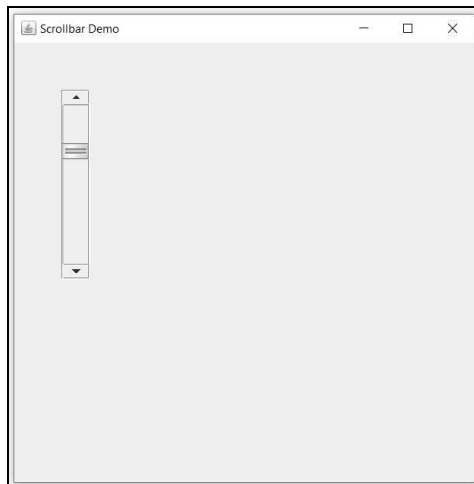
- The JScrollPane class is used to implement the horizontal and vertical scrollbars. This class is inherited from JComponent class.
- The commonly used constructors are :

JScrollbar()	Creates a vertical scrollbar with the initial values.
JScrollbar(int orientation)	Creates a scrollbar with the specified orientation and the initial values.
JScrollbar(int orientation, int value, int extent, int min, int max)	Creates a scrollbar with the specified orientation, value, extent, minimum and maximum.

- Following is a simple java program that displays the scrollbar component.

```
import javax.swing.*;
class ScrollBarDemo
{
    ScrollBarDemo()
    {
        JFrame fr= new JFrame("Scrollbar Demo");
        JScrollBar sb=new JScrollBar();
        sb.setBounds(50,50,30,200);
        fr.add(sb);
        fr.setSize(500,500);
        fr.setLayout(null);
        fr.setVisible(true);
    }
    public static void main(String args[])
    {
        new ScrollBarDemo();
    }
}
```

Output



Difference between Scrollbar and ScrollPane :

- 1) Scrollbar is a component whereas scrollPane is container.
- 2) Scrollbar cannot handle its own events whereas the ScrollPane can handle its events.
- 3) Scrollbar can not contain ScrollPane but scrollPane can have scrollbar.

3.4.13 Menus

- The menu bar is the most standard GUI which is present in almost all the applications.

- We can create a menu bar which contains several menus.
- Each menu can have several menu items.
- The separator can also be used to separate out these menus.
- Various methods and APIs that can be used for creating the Menus in Swing are -

JMenuBar

This class creates a menu bar. A menu bar normally contains several menus.

JMenu(String *str*)

This class creates several menus. The menus contain the menu items. The string *str* denotes the names of the menus.

JMenuItem(String *str*)

The menu items are the parts of menus. The string *str* denotes the names of the menu items.

setMnemonic(char *ch*)

The mnemonic key can be set using this method. The character which is passed to it as an argument becomes the mnemonic key. Hence using alt + ch you can select that particular menu.

JSeparator

This is the constructor of the class JSeparator which adds separating line between the menu items.

setJMenuBar

- This method is used to set menu bar to a specific frame.
- In the following Java program, we have created a frame on which a menu bar is placed. The menu bar contains three menus - File, Edit and Help. Each of these menus have their own set of menu items. The separators and mnemonic keys are also used in this program.

Java Program

```
import java.awt.*;
import javax.swing.*;
class MenuProg
{
    public static void main(String[] args)
    {
        JMenu menu1=new JMenu("File"); ← Creating Menu
        menu1.setMnemonic('F');
        menu1.add(new JMenuItem("New"));
        menu1.add(new JMenuItem("Open"));
        menu1.add(new JMenuItem("Save"));
        menu1.add(new JMenuItem("Save as..."));
    }
}
```

Adding menu items
to menus

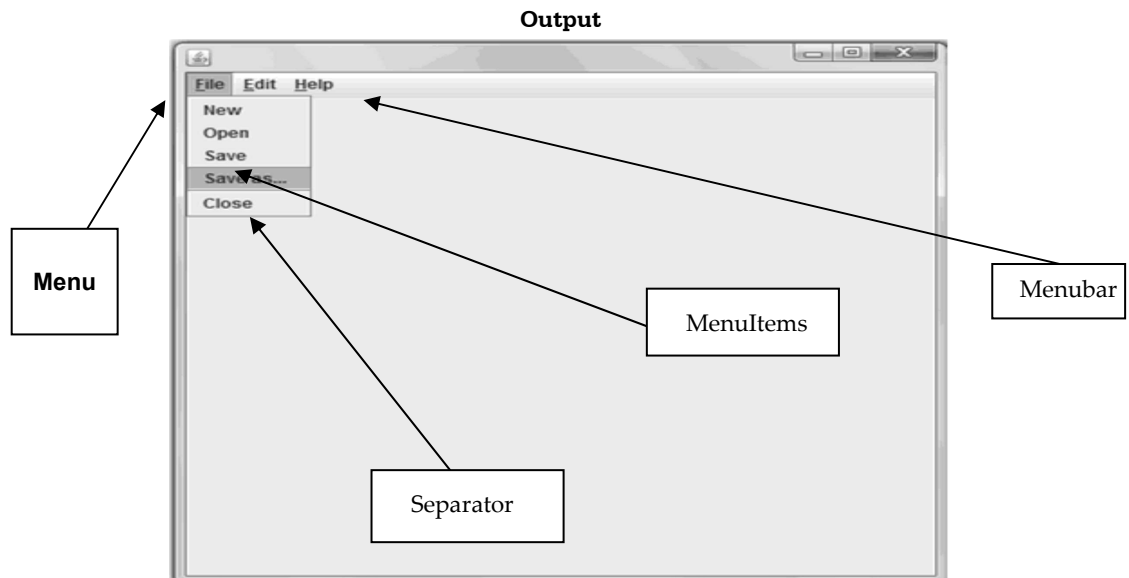
```
menu1.add(new JSeparator());
menu1.add(new JMenuItem("Close"));
JMenu menu2=new JMenu("Edit");
menu2.setMnemonic('E');
menu2.add(new JMenuItem("Undo"));
menu2.add(new JMenuItem("Redo"));
menu2.add(new JSeparator());
menu2.add(new JMenuItem("Cut"));
menu2.add(new JMenuItem("Copy"));
menu2.add(new JMenuItem("Paste"));

JMenu menu3=new JMenu("Help");
menu3.setMnemonic('H');
JMenuBar menubar=new JMenuBar(); ← Creating menubar
menubar.add(menu1);
menubar.add(menu2);
menubar.add(menu3);
```

} ← Adding menus to menubar

```
JFrame f=new JFrame(); ← Creating frames
f.setJMenuBar(menubar); ← placing menubar on the frame
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setVisible(true);
f.setSize(200,200);
```

```
}
```



In order to try out the mnemonic keys just press Alt+f key and the File menu gets selected.

Example 3.4.6 Write a program to create a frame with the following menus, such that the corresponding geometric object is created when a menu is clicked. i) Circle ii) Rectangle iii) Line iv) Diagonal for the rectangle.

Solution :

```
import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*;
class ShapesMenu extends JFrame implements ActionListener
{
    JMenuBar mb;
    JMenu menu;
    JMenuItem rect,line,oval;
    ShapesMenu()
    {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new FlowLayout());
        mb=new JMenuBar();

        menu=new JMenu("Shapes");
        mb.add(menu);

        rect=new JMenuItem("Rectangle");
        rect.addActionListener(this);
        menu.add(rect);

        line=new JMenuItem("Line");
        line.addActionListener(this);
        menu.add(line);

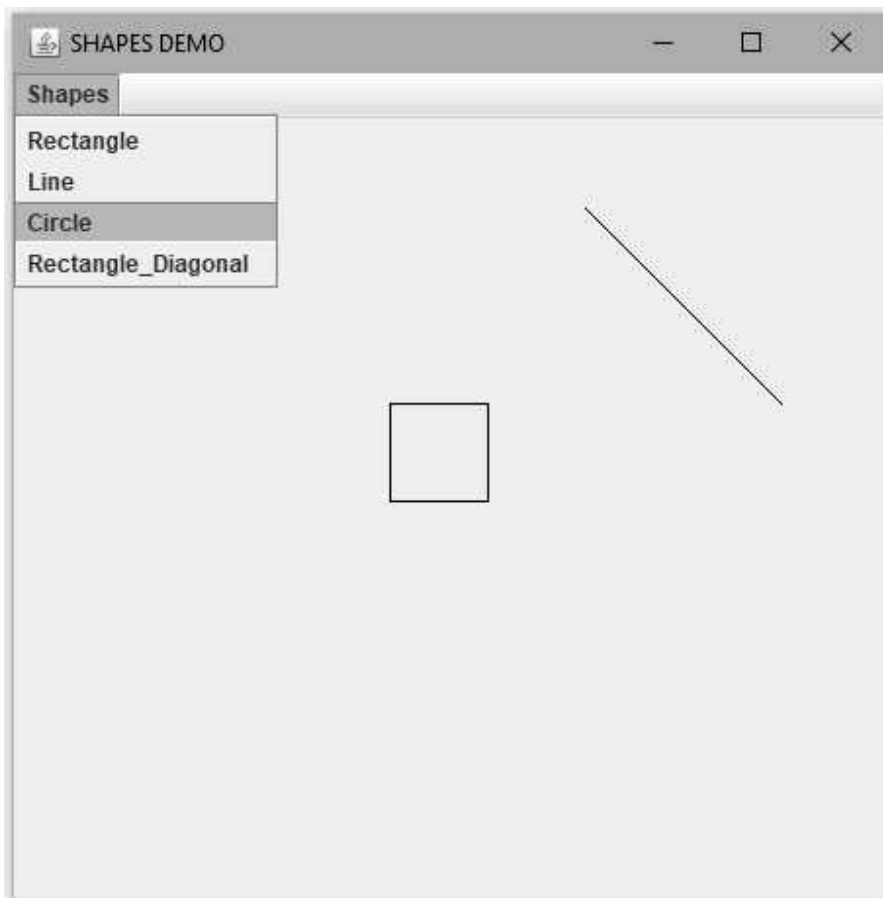
        oval=new JMenuItem("Circle");
        oval.addActionListener(this);
        menu.add(oval);

        line=new JMenuItem("Rectangle_Diagonal");
        line.addActionListener(this);
        menu.add(line);

        setJMenuBar(mb);
    }
    public void actionPerformed(ActionEvent ae)
    {
        String str=ae.getActionCommand();
        Graphics g=getGraphics();
```



```
        if(str=="Rectangle")
            g.drawRect(200,200,50,50);
        if(str=="Line")
            g.drawLine(300,100,400,200);
        if(str=="Circle")
            g.drawOval(200,300,100,100);
        if(str=="Rectangle_Diagonal")
            g.drawLine(200,200,250,250);
    }
    public static void main(String args[])
    {
        ShapesMenu f=new ShapesMenu();
        f.setTitle("SHAPES DEMO");
        f.setSize(500,500);
        f.setVisible(true);
    }
}
```

Output

Example 3.4.7 Create a simple menu application that enables a user to select one of the following items :

Radio 1

Radio 2

Radio 3

Radio 4

Radio 5

Red Dragon Radio

i) From the menu bar of the application ii) From a pop up menu iii) From a toolbar.

Add tooltips to each menu item that indicates some information about the Radio station such as type of music and its broadcast frequency.

Solution :

```
import java.awt.event.*;
import java.awt.*;
import javax.swing.*;

public class MenuApplication
{
    public static void main(String args[])
    {
        JFrame f = new JFrame("Simple Menu Application");
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        /* Creating Menu Bar */
        JMenuBar bar = new JMenuBar();
        JMenu menu = new JMenu("Options");
        menu.setMnemonic(KeyEvent.VK_O);
        ButtonGroup group = new ButtonGroup();
        JRadioButtonMenuItem menuItem = new JRadioButtonMenuItem("Radio 1");
        menuItem.setToolTipText("Music Masala 94.5KHz");
        group.add(menuItem);
        menu.add(menuItem);

        menuItem = new JRadioButtonMenuItem("Radio 2");
        menuItem.setToolTipText("Retro Melody 100.1KHz");
        group.add(menuItem);
        menu.add(menuItem);

        menuItem = new JRadioButtonMenuItem("Radio 3");
        menuItem.setToolTipText("Classical Khajana 98.3KHz");
        group.add(menuItem);
        menu.add(menuItem);
    }
}
```

```
menuItem = new JRadioButtonMenuItem("Radio 4");
menuItem.setToolTipText("Music on Demand 91.1KHz");
group.add(menuItem);
menu.add(menuItem);

menuItem = new JRadioButtonMenuItem("Radio 5");
menuItem.setToolTipText("Disco Station 95.3KHz");
group.add(menuItem);
menu.add(menuItem);

menuItem = new JRadioButtonMenuItem("Red Dragon Radio");
group.add(menuItem);
menu.add(menuItem);

bar.add(menu);
f.setJMenuBar(bar);

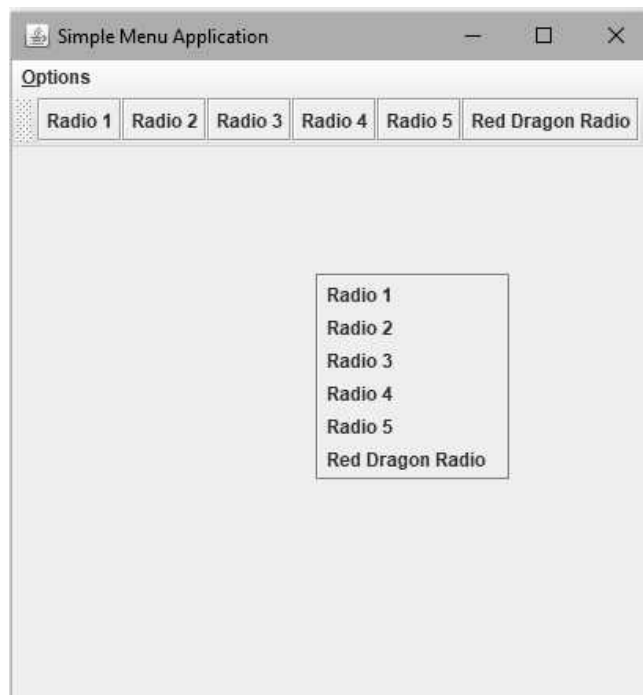
/* CreatingToolBar */
JToolBar toolbar = new JToolBar();
toolbar.setRollover(true);
JButton button = new JButton("Radio 1");
toolbar.add(button);
button = new JButton("Radio 2");
toolbar.add(button);
button = new JButton("Radio 3");
toolbar.add(button);
button = new JButton("Radio 4");
toolbar.add(button);
button = new JButton("Radio 5");
toolbar.add(button);
button = new JButton("Red Dragon Radio");
toolbar.add(button);
Container contentPane = f.getContentPane();
contentPane.add(toolbar, BorderLayout.NORTH);

/* Creating Popup menu */
JPopupMenu popup = new JPopupMenu();
JMenuItem mItem=new JMenuItem("Radio 1");
popup.add(mItem);
mItem=new JMenuItem("Radio 2");
popup.add(mItem);
mItem=new JMenuItem("Radio 3");
popup.add(mItem);
mItem=new JMenuItem("Radio 4");
popup.add(mItem);
mItem=new JMenuItem("Radio 5");
popup.add(mItem);
mItem=new JMenuItem("Red Dragon Radio");
```

```
popup.add(mItem);

f.addMouseListener(new MouseAdapter() {
    @Override
    public void mousePressed(MouseEvent e) {
        showPopup(e);
    }
    @Override
    public void mouseReleased(MouseEvent e) {
        showPopup(e);
    }
    private void showPopup(MouseEvent e) {
        if (e.isPopupTrigger()) {
            popup.show(e.getComponent(),
                e.getX(), e.getY());
        }
    }
});
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setSize(500, 500);
f.setVisible(true);
}
```

Output





3.4.14 Dialog Boxes

- The dialog box is useful to display some useful messages to the user. Using Swing we can create three types of dialog boxes -
 1. Simple message dialog box
 2. Confirm message dialog box
 3. Input dialog box
- These dialog boxes are created using **JOptionPane** class. This class is present in the **javax.swing.*** package. Hence we must import this file at the beginning.

Let us understand these dialog boxes with the help of programming examples -

Simple message dialog box

- This type of message box simply displays the informative message to the user.
- It has only OK button.

- It is expected that after reading out the message the user must click the OK button to return.
- The **JOptionPane** class provides the method **showMessageDialog()** in order to display the simple message box.
- Following is the simple Java program which shows how to create simple message dialog box -

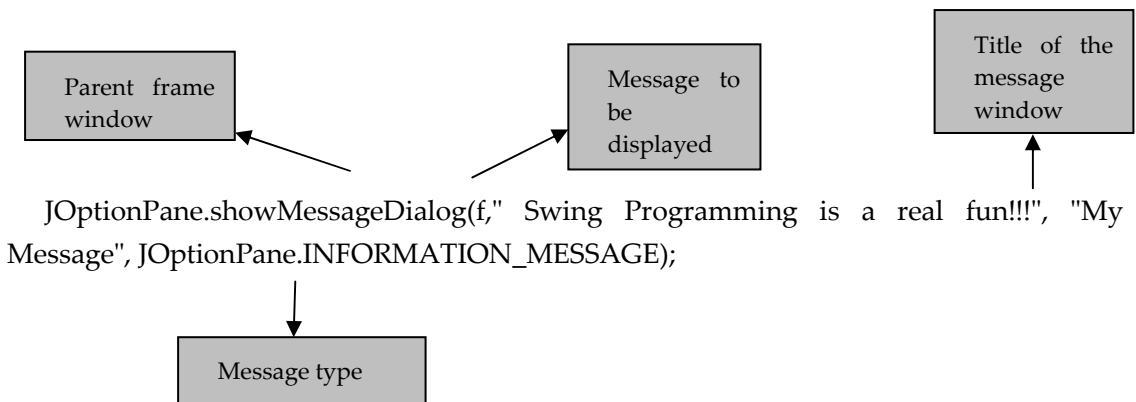
Java Program[DialogBox1Prog.java]

```
import java.awt.*;
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;
public class DialogBox1Prog
implements ActionListener
{
    JFrame f;
    public static void main(String[] args)
    {
        new DialogBox1Prog();
    }
    public DialogBox1Prog()
    {
        f=new JFrame("Dialog Box Demo");
        JButton B=new JButton("Click Me!!");
        Container container=f.getContentPane();
        container.setLayout(new FlowLayout());
        container.add(B);
        B.addActionListener(this);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setSize(300,300);
        f.setVisible(true);
    }
    public void actionPerformed(ActionEvent e)
    {
        JOptionPane.showMessageDialog(f,"Swing Programming is a real fun!!!","My
        Message",JOptionPane.INFORMATION_MESSAGE);
    }
}
```

Output**Program explanation**

In above Java program, we have created one frame window, on which one push button is placed. When we click the button, then the simple message box will be displayed. This message box has only one OK button.

To display the dialog box we have

**Confirm message dialogbox**

- This type of dialog box will display some message to the user and will get the confirmation from him.
- User can give his confirmation either by clicking OK, CANCEL, YES or NO button.

- The confirm message dialog box will display the message either along with the OK and CANCEL button or with the YES, NO or CANCEL button.
- In the following Java program we have used three types of message boxes - simple message box, confirm message box with OK and CANCEL and the confirm message box with YES, NO and CANCEL.

Java Program[DialogBox2Prog.java]

```
import java.awt.*;
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;
public class DialogBox2Prog
implements ActionListener
{
    JFrame f;
    public static void main(String[] args)
    {
        new DialogBox2Prog();
    }
    public DialogBox2Prog()
    {
        f=new JFrame("Dialox Box Demo");
        Container container=f.getContentPane();
        container.setLayout(new FlowLayout());
        JButton B=new JButton("Simple Message DialogBox");
        container.add(B);
        B.addActionListener(this);

        B=new JButton("OK and Cancel DialogBox");
        container.add(B);
        B.addActionListener(this);

        B=new JButton("Yes/No/Cancel DialogBox");
        container.add(B);
        B.addActionListener(this);

        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setSize(300,300);
        f.setVisible(true);
    }
    public void actionPerformed(ActionEvent e) //event handler function
```

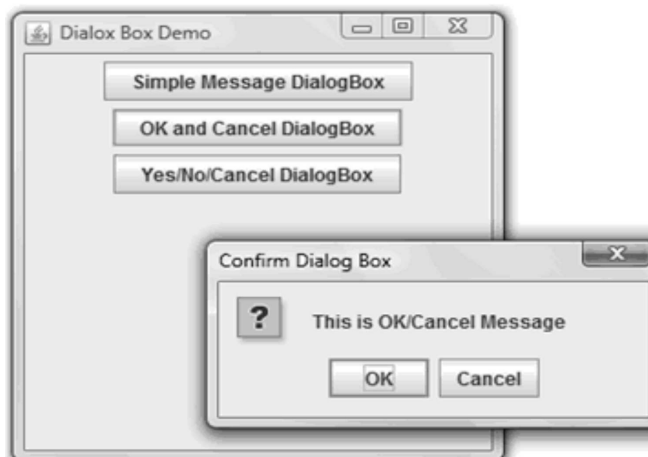
On this button click simple message box will be displayed

On this button click OK/Cancel dialog box will be displayed

On this button click Yes/No/Cancel dialog box will be displayed


```
{
String s=e.getActionCommand(); //getting the string present on the button
if(s.equals("Simple Message DialogBox"))
    JOptionPane.showMessageDialog(f,"Simple Message");
else if(s.equals("OK and Cancel DialogBox"))
{
    if(JOptionPane.showConfirmDialog(f,"This is OK/Cancel Message","Confirm Dialog
        Box",JOptionPane.OK_CANCEL_OPTION)==0)
        JOptionPane.showMessageDialog(f,"You have clicked OK button");
    else
        JOptionPane.showMessageDialog(f,"You have clicked Cancel button");
}
else if(s.equals("Yes/No/Cancel DialogBox"))
{
    if(JOptionPane.showConfirmDialog(f,"This is Yes/No/Cancel Message","Confirm Dialog
        Box",JOptionPane.YES_NO_CANCEL_OPTION)==0)
    {
        JOptionPane.showMessageDialog(f,"You have clicked Yes button");
    }
    else
    {
        JOptionPane.showMessageDialog(f,"You have clicked OK or Cancel button");
    }
}
}
}
}
```

Output



Program explanation

For the confirm dialog box, the option type could be `OK_CANCEL`, `YES_NO_OPTION`, `OK_OPTION`, `NO_OPTION` which are associated with some constant values.

In above program, when the user clicks the button for OK and Cancel message then appropriate message will be displayed. If user clicks the OK button then the message "You have clicked OK button will be displayed" otherwise the message "You have clicked Cancel button" will be displayed.

3.4.15 Tabbed Pane

Tabbed pane is a type of component in which group of folders can be represented together and particular folder can be selected from the tab.

Each folder has a title and when the user selects the particular folder on the tab then only it will be displayed. One folder can be displayed at a time.

For displaying the tabbed pane there exists a **JTabbedPane** class which extends the **JComponent** class.

Using the **addTab** method the folders can be added to the tabbed pane. The syntax for this method is

```
void addTab(String str, Component comp)
```

The *str* represents the string which will be displayed as a title to the tab.

The *comp* is a reference to the component which should be added to the tabbed pane.

Following program illustrates the use of tabbed pane.

Step 1 : Create an applet.

Step 2 : Create the tabbed pane.

Step 3 : Create a contentpane.

Step 4 : Place the tabbed pane on the content pane.

Step 5 : Define the components of tabbed pane.

Java program

```
import javax.swing.*;
/*
<applet code="TabbedPaneDemo" width=500 height=400>
</applet>
*/

public class TabbedPaneDemo extends JApplet
```

```
{
public void init()
{
    JTabbedPane mytpane = new JTabbedPane();
    mytpane.addTab("Laptop", new LaptopPanel());
    mytpane.addTab("OS", new OSPanel());
    mytpane.addTab("Database", new DatabasePanel());
    mytpane.addTab("Languages", new LangPanel());
    getContentPane().add(mytpane);
}
}
```

Adding the folders on the
tabbed pane

```
class LaptopPanel extends JPanel
```

Languages Component defined

```
{
    public LaptopPanel()
    {
        JRadioButton jrb1 = new JRadioButton("DELL");
        add(jrb1);
        JRadioButton jrb2 = new JRadioButton("Samsung");
        add(jrb2);
        JRadioButton jrb3 = new JRadioButton("HP");
        add(jrb3);
        JRadioButton jrb4 = new JRadioButton("Acer");
        add(jrb4);
        ButtonGroup bg=new ButtonGroup();
        bg.add(jrb1);
        bg.add(jrb2);
        bg.add(jrb3);
        bg.add(jrb4);
    }
}
```

```
class OSPanel extends JPanel
```

OS Component defined

```
{
    public OSPanel()
    {
        JComboBox jcb = new JComboBox();
        jcb.addItem("Windows XP");
        jcb.addItem("Windows Vista");
        jcb.addItem("Windows 7");
        jcb.addItem("Ubuntu");
        add(jcb);
    }
}
```

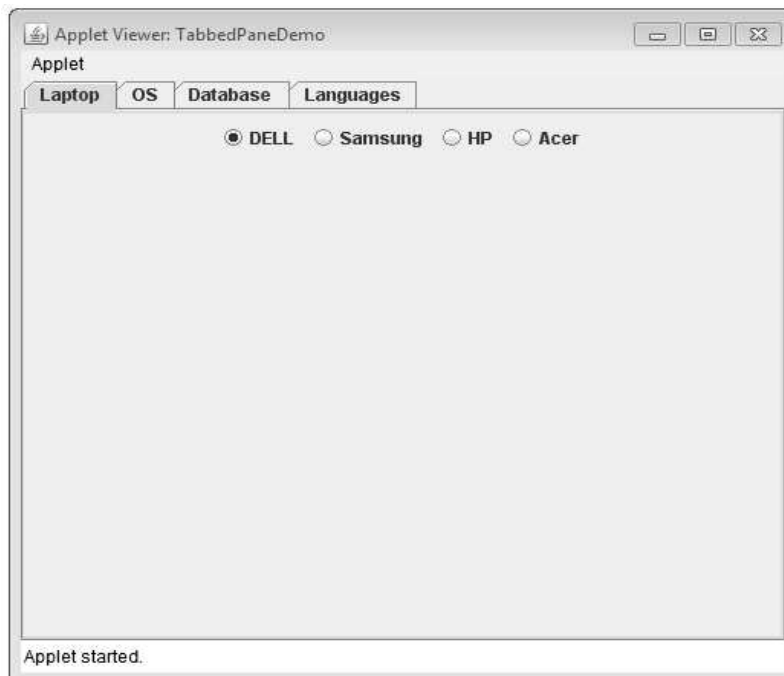
```
class DatabasePanel extends JPanel
```

Database Component defined

```
{
public DatabasePanel()
{
    JCheckBox cb1 = new JCheckBox("Oracle");
    add(cb1);
    JCheckBox cb2 = new JCheckBox("MySQL");
    add(cb2);
    JCheckBox cb3 = new JCheckBox("Microsoft Access");
    add(cb3);
}
}
class LangPanel extends JPanel
{
    public LangPanel()
    {
        JButton b1 = new JButton("JSP");
        add(b1);
        JButton b2 = new JButton("ASP");
        add(b2);
        JButton b3 = new JButton("PHP");
        add(b3);
    }
}
```

Languages Component defined

Output



3.4.16 JTree

Tree is a type of component that gives the hierarchical view of data. User can expand or shrink the nodes of the tree. In swing the trees are implemented using the **JTree** class. This class extends the **JComponent**.

The most commonly used constructor for this class is -

```
JTree(TreeNode root)
```

The *root* represents the root node of the tree.

Using the **DefaultMutableTreeNode**, it creates a tree node with no root node, the child of root node, specified by user object and it allows only children that have to be specified. It takes boolean types values either '**true**' or '**false**'. If you will take '**true**' that means children node are allowed.

Step 1 : Make use of applet for implementation of the tree program.

Step 2 : Create a tree with root, child and grand child nodes.

Step 3 : Create a content pane object.

Step 4 : Add the JTree component on the content pane.

Java Program

```
import javax.swing.*;
import java.awt.*;
import javax.swing.tree.*;
/*
<applet code="TreeDemo" width=300 height=200>
</applet>
*/

public class TreeDemo extends JApplet
{
    public void init()
    {
        Container contentPane=getContentPane();
        contentPane.setLayout(new BorderLayout());

        DefaultMutableTreeNode root = new DefaultMutableTreeNode("Root", true);

        DefaultMutableTreeNode c1 = new DefaultMutableTreeNode("Child 1");
        root.add(c1);

        DefaultMutableTreeNode c2 = new DefaultMutableTreeNode("Child 2");
        root.add(c2);
    }
}
```

```
DefaultMutableTreeNode gc1 = new DefaultMutableTreeNode("GrandChild 1");
DefaultMutableTreeNode gc2 = new DefaultMutableTreeNode("GrandChild 2");
DefaultMutableTreeNode gc3 = new DefaultMutableTreeNode("GrandChild 3");
c2.add(gc1);
c2.add(gc2);
c2.add(gc3);

DefaultMutableTreeNode c3 = new DefaultMutableTreeNode("Child 3");
root.add(c3 );

DefaultMutableTreeNode c4 = new DefaultMutableTreeNode("Child 4");
root.add(c4);

DefaultMutableTreeNode c5 = new DefaultMutableTreeNode("Child 5");
root.add(c5);

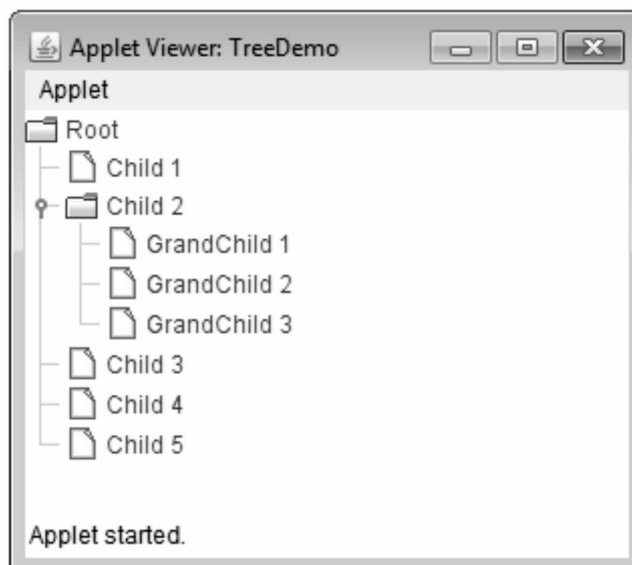
JTree tree = new JTree(root);
contentPane.add(tree);
}
}
```

Following commands can be used to execute the above code.

```
D:\JavaPrograms>javac TreeDemo.java
```

```
D:\JavaPrograms>AppletViewer TreeDemo.java
```

Output



3.4.17 JTable

Table is a component that arranges the data in rows and columns. The **JTable** class extends the **JComponent** class. The constructor used for table component is -

```
JTable( object[][] tablevalues object [] columnheader)
```

The *tablevalues* indicate the data that can be arranged in tabular fashion.

The *columnheader* denotes the header for each column.

Following is a simple program that shows the use of JTable component.

Java Program

```
import javax.swing.*.*;
import java.awt.*.*;
import javax.swing.tree.*.*;
/*
<applet code="TableDemo" width=300 height=100>
</applet>
*/

public class TableDemo extends JApplet
{
    public void init()
    {
        Container contentPane = getContentPane();

        contentPane.setLayout(new BorderLayout());

        final String[] th = { "Name", "City", "Salary", "Designation" };
        final Object[][] mytable = {
            { "Arun", "Pune", "5000", "Accountant" },
            { "Archana", "Mumbai", "7000", "Executive" },
            { "Shivani", "Bangalore", "10000", "Manager" },
            { "Priyanka", "Chennai", "8000", "Programmer" },
            { "Monika", "Hyderabad", "10000", "Designer" },
            { "Shilpa", "Hyderabad", "12000", "Director" },
            { "Anuja", "Delhi", "17000", "Director" },
            { "Kumar", "Pune", "10000", "Manager" }
        };

        JTable table = new JTable(mytable, th);

        int vscrollbar = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
        int hscrollbar = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
```

```

JScrollPane mypane = new JScrollPane(table, vscrollbar, hscrollbar);
contentPane.add(mypane, BorderLayout.CENTER);
}
}

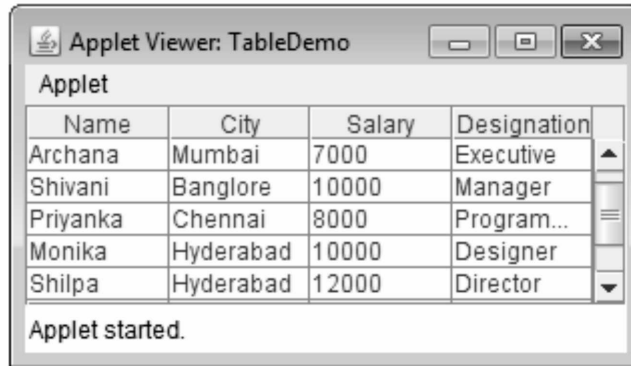
```

Following commands are used to display the output.

```
D:\JavaPrograms>javac TableDemo.java
```

```
D:\JavaPrograms>AppletViewer TableDemo.java
```

Output



Example 3.4.8 Write a Java program to display the 3×3 magic square using `JTable`.

Solution :

```

import javax.swing.*.*;
import java.awt.*.*;
import javax.swing.table.*;
/*
<applet code="TableDemo" width=100 height=100>
</applet>
*/

public class TableDemo extends JApplet
{
    public void init()
    {
        Container contentPane = getContentPane();

        contentPane.setLayout(new BorderLayout());
        final String[] th = { "", "", "" };
        final Object[][] mytable = {
            { "8", "1", "6" },
            { "3", "5", "7" },
            { "4", "9", "2" }
        }
    }
}

```



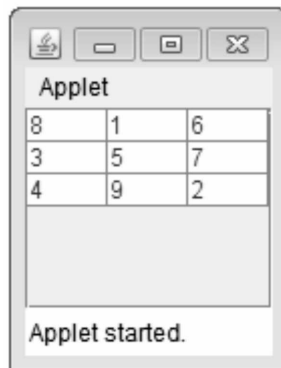
```
};

JTable table = new JTable(mytable,th);

int vscrollbar = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int hscrollbar = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;

JScrollPane mypane = new JScrollPane(table, vscrollbar, hscrollbar);
contentPane.add(mypane, BorderLayout.CENTER);
}
}
```

Output



3.5 Java Utilities (java.util Package)

Package java.util contains the collections framework, legacy collection classes, event model, date and time facilities, internationalisation and miscellaneous utility classes.

Importing java.util package

The import is a java keyword which is used for importing a Java class or entire Java package. For example import java.util.Calendar; means you are importing a single calendar class. If you want to import all the classes from any Java package, your import statement must be like this **import java.util.***. That means you are importing the entire java.util package.

3.6 The Collection Framework

- The standard data structures can be implemented in Java using some library classes and methods.
- These classes are present in the **java.util** package.
- The collection framework is comprised of **collection classes** and **collection interfaces**.

- Basically collection is a group of objects which are designed to perform certain task. These tasks are associated with alteration of data structures.
- The collection classes are the group of classes used to implement the collection interfaces.
- Various collection classes are

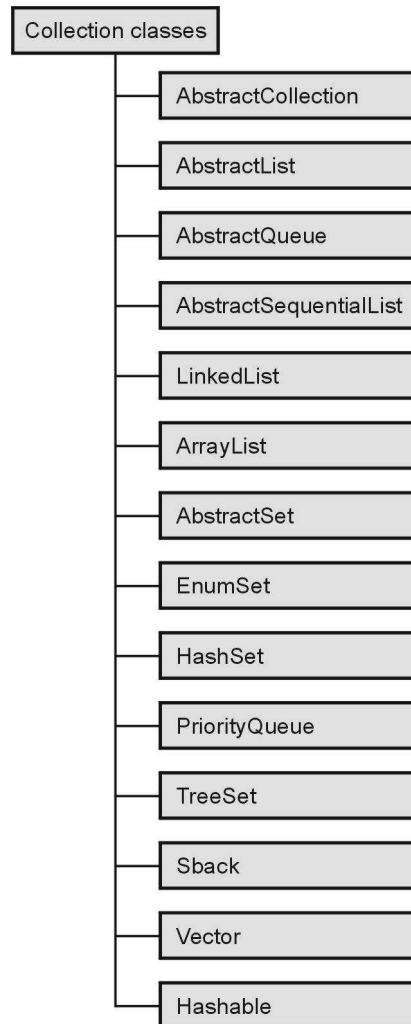


Fig. 3.6.1 Collection class hierarchy

Review Question

1. Write short note on - collection framework.

3.7 Collections of Objects and Types

Collection classes provide the implementation of different collection interfaces.

Fig. 3.7.1 shows different collection interfaces.

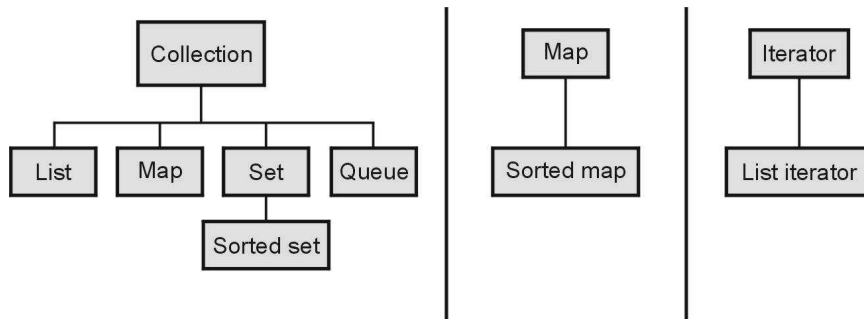


Fig. 3.7.1 Interfaces in collection framework

The collection interfaces contain several useful methods by which we can modify the collections. Various methods that are supported by collection are as given below -

Method	Description
boolean add(Object obj)	Objects are added using this method. This method takes arguments of type object.
boolean addAll(Collection collection)	Entire content of one collection can be added to another using this method.
void clear()	In order to clear the collection, this method is used.
boolean contains(Object obj)	For checking whether the collection contains specific object or not this method is used.
boolean containsAll(Collection collection)	If all the elements of the collection are present in the collection then this method returns true.
boolean isEmpty()	It's a Boolean method which determines whether collection is empty or not. If a collection is empty then it return true otherwise false.
Iterator iterator()	Returns iterator to the collection.
boolean remove(Object obj)	An object can be removed by this method.
boolean removeAll(Collection collection)	It helps in removing a group of objects.
int size()	It returns the total number of elements in the collection.

Object[] toArray()	Returns an array of elements to the invoking collection. Basically these array elements are the copies of the collection that calls the toArray method.
boolean equals(Object obj)	For comparing two collections this method is used.

The methods in collection interface throws has two common exceptions -

Unsupported Operation Exception for illegal supporting operation and **ClassCastException** when one object is incompatible with other object.

List Interface

The List interface extends the Collection interface. It is basically the sequence of elements in which the elements can be inserted and accessed by their position in the list. The duplicate elements are allowed in this interface. Various methods used in the **List** interface are enlisted in following table -

Method	Description
void add(int i, Object obj)	This method inserts the object obj at specified location in the list. This location is denoted by index i.
void add(int i, Collection collection)	This method inserts elements of collection collection at specified location in the list. This location is denoted by index i.
Object get(int i)	To get the object stored at specified location in the list, this method is used. This location can be obtained with the help of index i.
Object set(int i, Object obj)	The value of the element can be set using this method.
int indexOf(Object obj)	If object is an element of the list then its index will be returned by this method.
int lastIndexOf(Object obj)	This method returns the index of the last object present in the list.
ListIterator listIterator()	It returns an iterator of the element to the start of the list.
ListIterator listIterator(int i)	It returns an iterator to the list which begins at specified index.

Object remove(int i)	This method is for removing the element present at the specific position.
List subList(int starting,int end)	It returns the sublist specified within the starting index and ending index.

Set Interface

The set interface is used to define the set of elements. It extends the collection interface. This interface defined unique elements. Hence if any duplicate elements is tried to insert in the set then the add() method returns false.

SortedSet Interface

This interface is inherited from the set interface and allows the elements to be arranged in ascending order. The methods defined in this interface normally throw the exception such as **NoSuchElementException**, **NullPointerException** and **ClassCastException**.

Various methods used by this interface are as given below -

Method	Description
Comparator comparator()	This method returns the comparator object. If the elements in the SortedSet are present in ascending order then this method returns null.
Object first()	It returns the first element present in the SortedSet.
Object last()	It returns the last element present in the SortedSet.
SortedSet headset(Object target)	This method returns all those elements which are less than the target element.
SortedSet subset(Object source,Object target)	This method returns all those elements of SortedSet which are present between source and target.
SortedSet headset(Object target)	This method returns all those elements which are greater than or equal to the target element.

Map Interface

This interface maps a unique key element to the value. Thus map interface represents a key-value pair.

SortedMap Interface

The **SortedMap** is inherited from the Map interface. In this interface the elements are stored in ascending order. This sorted order is based on the key.

Review Question

1. Explain the collection interface with example.

3.8 List Interface

- Lists interface extends the **collection** interface.
- It is basically the sequence of elements in which the elements can be inserted and accessed by their position in the list. The duplicate elements are allowed in this interface.
- There are two concrete classes for defining the **List** interface.

1. ArrayList

2. LinkedList

3.8.1 ArrayList

- The **ArrayList** class implements the List interface. It is used to implement the dynamic array.
- The dynamic array means the size of array can be created as per requirement. Hence **ArrayList** is a variable length array of object references.
- Initially **ArrayList** is created with some initial size and then as per requirement we can extend the size of array. When the objects are removed then the size of array can be reduced.
- The syntax of using **ArrayList** is as given below -

ArrayList() ← Creates an empty list

ArrayList(Collection collection) ← Creates a list in which the collection elements are added

ArrayList(int c) ← Creates a list with specified capacity c, the capacity represents the size of the underlying array

Let us see simple Java program which demonstrates the **ArrayList**

Java Program [ArrayListProg.Java]

```
/******  
Program uses the ArrayList collection class to  
implement ArrayList data structure  
*****/  
import java.util.*;  
class ArrayListProg  
{  
    public static void main(String[] args)  
    {  
        System.out.println("\n\t\t Program for Implementing Array List");  
    }  
}
```

```
ArrayList obj=new ArrayList();           //creation of ArrayList
System.out.println("\n Inserting some elements in the array");
obj.add(10);
obj.add(20);
obj.add(30);
obj.add(40);
obj.add(50);
System.out.println("The array elements are... "+obj);
System.out.println("\n Inserting some elements in the array in between");
obj.add(4,45);                           //inserting after element 40
System.out.println("The array elements are... "+obj);
System.out.println("\n Removing some elements from the array");
obj.remove(1);                            //array index is passed to remove method
System.out.println("The array elements are... "+obj);
System.out.println("The array size is... "+obj.size());
}
}
```

Output

Program for Implementing Array List

```
Inserting some elements in the array
The array elements are... [10, 20, 30, 40, 50]

Inserting some elements in the array in between
The array elements are... [10, 20, 30, 40, 45, 50]

Removing some elements from the array
The array elements are... [10, 30, 40, 45, 50]
The array size is... 5
```

Program explanation

In above program we have created an array list of integer numbers we can also pass some character elements to the array list. While removing the element from the list we should pass the index position of the element. Note that the elements are placed from 0th index in the ArrayList. The above array is a dynamic array. As we insert the elements it grows and as we remove the elements it gets shrunk.

Example 3.8.1 *Write a program which stores the list of strings in an ArrayList and then displays the contents of the list.*

Solution :

```
import java.util.*;
class ArrayListProg
{
```

```

public static void main(String[] args)
{
    System.out.println("\n\t\t Program for Implementing Array List for List of Strings");
    ArrayList obj=new ArrayList();           //creation of ArrayList
    System.out.println("\n Inserting some elements in the array");
    obj.add("AAA");
    obj.add("BBB");
    obj.add("CCC");
    obj.add("DDD");
    obj.add("EEE");
    System.out.println("The array elements are... "+obj);
    System.out.println("The array size is... "+obj.size());
}
}

```

Output

```

Program for Implementing Array List for List of Strings
Inserting some elements in the array
The array elements are... [AAA, BBB, CCC, DDD, EEE]
The array size is... 5

```

3.8.2 LinkedList

- Linked List is a collection of nodes in which every node contains two fields Data and Next pointer fields.
- The link list can be graphically represented as follows -

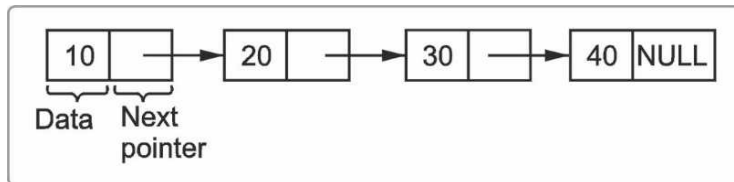


Fig. 3.8.1 Linked list

- The java.util package provides the collection class LinkedList in order to implement the List interface.
- The syntax of using this class is

LinkedList() ← creates an empty linked list

LinkedList(Collection collection) ← creates a linked list having the elements of *collection*

Programming example :

Following program makes use of various methods such as **add()**, **remove()**, **addFirst()**, **addLast()**, **removeFirst()**, **removeLast()** for manipulating the contents of linked list.


```
import java.io.*;
import java.util.*;
class LinkedListProg
{
    public static void main(String[] args)throws IOException
    {
        char ans='y',ch='y';
        int choice,val,position;
        String str;
        LinkedList obj=new LinkedList();
        Scanner s=new Scanner(System.in);
        do
        {
            System.out.println("\n\t\t Program for Implementing Linked List");
            System.out.print("\n1.Create\n2.Display \n3.Insert First\n4. Insert Last");
            System.out.print("\n5.Delete First\n6.Delete Last");
            System.out.print("\n7.Insert At any Position");
            System.out.println("\n8.Delete From any Position");
            System.out.println("Enter Your choice");
            choice=s.nextInt();
            switch(choice)
            {
                case 1:
                    do
                    {
                        System.out.println("\n Enter the element to be inserted in the list");
                        val=s.nextInt();
                        obj.add(val);
                        System.out.println(" Do u want to insert more elements?");
                        str =s.next();
                        ans=str.charAt(0);

                    }while(ans=='y');
                    break;
                case 2:
                    System.out.println("\t The List elements are... "+obj);
                    System.out.println("\t The size of linked list is... "+obj.size());
                    break;
                case 3: System.out.println("\n Enter the element to be inserted in the list");
                        val=s.nextInt();
                        obj.addFirst(val);
                        System.out.println("\n The element inserted!!!");
                        break;
                case 4: System.out.println("\n Enter the element to be inserted in the list");
```

```
        val=s.nextInt();
        obj.addLast(val);
        System.out.println("\n The element inserted!!!");
        break;
    case 5: obj.removeFirst();
        System.out.println("\n The element deleted!!!");
        break;
    case 6: obj.removeLast();
        System.out.println("\n The element deleted!!!");
        break;
    case 7: System.out.println("\n Enter the element to be inserted in the list");
        val=s.nextInt();
        System.out.println("\n Enter the position at which the element is to be inserted");
        position=s.nextInt();
        obj.add(position,val);
        System.out.println("\n The element inserted!!!");
        break;
    case 8: System.out.println("\n Enter the position of element to be deleted");
        position=s.nextInt();
        obj.remove(position);
        System.out.println("\n The element deleted!!!");
        break;
    }
    System.out.println("\n Do u want to go to main menu?");
    str=s.next();
    ch=str.charAt(0);
    }while(ch != 'y');
}
}
```

Output

Program for Implementing Linked List

```
1. Create
2. Display
3. Insert First
4. Insert Last
5. Delete First
6. Delete Last
7. Insert At any Position
8. Delete From any Position
Enter your choice
1
Enter the element to be inserted in the list
10
Do u want to insert more elements?
```

```
y
Enter the element to be inserted in the list
20
Do u want to insert more elements?
y
Enter the element to be inserted in the list
30
Do u want to insert more elements?
y
Enter the element to be inserted in the list
40
Do u want to insert more elements?
n
Do u want to go to main menu?
y
        Program for Implementing Linked List
1.Create
2.Display
3.Insert First
4. Insert Last
5.Delete First
6.Delete Last
7.Insert At any Position
8.Delete From any Position
Enter Your choice
2
        The List elements are... [10, 20, 30, 40]
        The size of linked list is... 4
Do u want to go to main menu?
y
        Program for Implementing Linked List
1.Create
2.Display
3.Insert First
4. Insert Last
5.Delete First
6.Delete Last
7.Insert At any Position
8.Delete From any Position
Enter Your choice
3
Enter the element to be inserted in the list
9
The element inserted!!!
```

```

Do u want to go to main menu?
y
    Program for Implementing Linked List
1.Create
2.Display
3.Insert First
4. Insert Last
5.Delete First
6.Delete Last
7.Insert At any Position
8.Delete From any Position
Enter Your choice
2
    The List elements are... [9, 10, 20, 30, 40]
    The size of linked list is... 5
Do u want to go to main menu ?

```

Difference between ArrayList and LinkedList

Sr. No.	ArrayList	LinkedList
1.	ArrayList internally uses dynamic array to store the elements.	LinkedList internally uses doubly linked list to store the elements.
2.	It internally uses array. If any element is removed from the array, all the bits are shifted in memory.	It uses doubly linked list so no bit shifting is required in memory.
3.	Manipulation with ArrayList is slow.	Manipulation with LinkedList is faster than ArrayList.
4.	ArrayList class can act as a list only because it implements List only.	LinkedList class can act as a list and queue both because it implements List and Deque interfaces.
5.	ArrayList is better for storing and accessing data.	LinkedList is better for manipulating data.

Review Question

1. What is the difference between ArrayList and LinkedList ?

3.9 Vector

Vector is similar to the **Array List** class which implements the **dynamic array**. It implements the list interface. This class contains various additional methods which are enlisted below -

Method	Description
void addElement(object)	For adding some element in the vector this method is used.
void insertElementAt(object obj,int pos)	For inserting the element in the vector specified by its position.
boolean removeElement(object ele)	This method removes the specified element.
void removeAllElements()	This method is for removing all the elements from the vector.
void removeElementAt(int pos)	The element specified by its position gets deleted from the vector.
int capacity()	It returns the capacity of the vector.
int size()	It returns the total number of elements present in the vector.
boolean isEmpty()	It returns true if the vector is empty.
object firstElement()	Returns the first element of the vector.
int indexOf(object ele)	It returns the index of the corresponding element in the vector.
void setSize(int size)	This method is for setting the size of the vector.

Following program illustrates the implementation of vector class.

Java Program [VectorProg.Java]

```

/*****
Program for implementing vector class
*****/
import java.io.*;
import java.util.*;
class VectorProg
{
    public static void main(String[] args)throws IOException
    {

```

```

Vector obj=new Vector(3);
int ival;
double dval;
char ans='y';
System.out.println("The capacity of vector is "+obj.capacity());
System.out.println("The total number of elements are "+obj.size());
System.out.println("\t Inserting the integers");
do
{
    System.out.println("\n Enter some integer value");
    ival=getInt();
    obj.addElement(ival);
    System.out.println("\n Do u want to enter more?");
    ans=getChar();
}while(ans=='y');
System.out.println("The elements in the vector are "+obj);
System.out.println("\t Inserting the double values");
do
{
    System.out.println("\n Enter some double value");
    dval=getDouble();
    obj.addElement(dval);
    System.out.println("\n Do u want to enter more?");
    ans=getChar();
}while(ans=='y');
System.out.println("The elements in the vector are "+obj);
System.out.println("\t The size of vector is "+obj.size());
System.out.println("\t The first element of the vector is "+(Integer)obj.firstElement());
System.out.println("\t The last element of the vector is "+(Double)obj.lastElement());
System.out.println("Removing 2 the elements");
obj.remove(0);
obj.remove(2);
System.out.println("Now The elements in the vector are "+obj);

}

////////////////////////////////////
//Following functions are used to handle the inputs entered
//by the user using keyboard
////////////////////////////////////
public static String getString() throws IOException
{
    InputStreamReader input = new InputStreamReader(System.in);
    BufferedReader b = new BufferedReader(input);
    String str = b.readLine();//reading the string from console

```

```
    return str;
}

public static char getChar() throws IOException
{
    String str = getString();
    return str.charAt(0);//reading first char of console string
}

public static int getInt() throws IOException
{
    String str = getString();
    return Integer.parseInt(str);//converting console string to numeric value
}

public static double getDouble() throws IOException
{
    String str = getString();
    return Double.parseDouble(str);//converting console string to numeric value
}
}
```

Output

The capacity of vector is 3
The total number of elements are 0
 Inserting the integers

Enter some integer value
11

Do u want to enter more?
y

Enter some integer value
22

Do u want to enter more?
y

Enter some integer value
33

Do u want to enter more?
y

Enter some integer value
44

```
Do u want to enter more?
n
The elements in the vector are [11, 22, 33, 44]
    Inserting the double values

Enter some double value
1.1

Do u want to enter more?
y

Enter some double value
2.22

Do u want to enter more?
y

Enter some double value
3.333

Do u want to enter more?
n
The elements in the vector are [11, 22, 33, 44, 1.1, 2.22, 3.333]
    The size of vector is 7
    The first element of the vector is 11
    The last element of the vector is 3.333
Removing 2 the elements
Now The elements in the vector are [22, 33, 1.1, 2.22, 3.333]
```

3.10 Set Interface

- The set interface is used to define the set of elements.
- It extends the collection interface.
- This interface defined unique elements. Hence if any duplicate elements is tried to insert in the set then the add() method returns false.
- Some commonly used functionalities in Set interface are -

Method	Description
add()	Adds an object to the collection.
clear()	Removes all objects from the collection.
contains()	Returns true if a specified object is an element within the collection.
isEmpty()	Returns true if the collection has no elements.

iterator()	Returns an Iterator object for the collection, which may be used to retrieve an object.
remove()	Removes a specified object from the collection.
size()	Returns the number of elements in the collection.

There are two commonly used ways of implementing set interface.

3.10.1 HashSet

- HashSet is a collection class that implements the set interface.
- It creates the hash table.
- The hash table is a data structure in which the data is stored using hashing function. Hence the elements get stored in the hash table based on the hash key returned by hash function.
- The syntax of using HashSet class is -

```
HashSet()  
HashSet(Collection collection)  
HashSet(int c)  
HashSet(int c,float fillRatio)
```

- The fillRatio ranges from 0.0 to 1.0. This parameter determines how full the Hash set can be before getting resized.
- Various methods supported by the Set class are supported by the HashSet class. Following program implements the hash table -

Java Program[HashSetProg.java]

```
import java.io.*;  
import java.util.*;  
class HashSetProg  
{  
    public static void main(String[] args)throws IOException  
    {  
        char ans='y';  
        int val;  
        System.out.println("\n\t\t Program to create Hash Set");  
        HashSet obj=new HashSet();  
        Scanner s=new Scanner(System.in);  
        System.out.println("\n Creation of hash set");  
        do  
        {  
            System.out.println(" Enter the element ");
```

```
    val=s.nextInt();
    obj.add(val);
    System.out.println("\n Do u want to enter more elements?");
    String str=s.next();
    ans=str.charAt(0);
}while(ans=='y');
    System.out.println("The elements are ..." +obj);
}
}
```

Output

```
        Program to create Hash Set
Creation of hash set
Enter the element
10
Do u want to enter more elements?
y
Enter the element
20
Do u want to enter more elements?
y
Enter the element
30
Do u want to enter more elements?
y
Enter the element
40
Do u want to enter more elements?
y
Enter the element
50
Do u want to enter more elements?
n
The elements are ...[50, 20, 40, 10, 30]
```

3.10.2 TreeSet

- The TreeSet class implements the **Set** collection class.
- This class is basically for creating the Tree data structure.
- The binary tree gets created and when we try to display the tree then we get the nodes arranged in sorted manner.
- The following program makes use of TreeSet collection class.

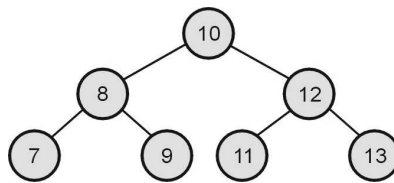


Fig. 3.10.1 Binary tree

- We will create following tree in the program -

Java Program[TreeSetProg.java]

```

import java.io.*;
import java.util.*;
class TreeSetProg
{
    public static void main(String[] args)throws IOException
    {
        char ans='y';
        int val;
        System.out.println("\n\t\t Program to create Tree data structure");
        TreeSet obj=new TreeSet();
        Scanner s=new Scanner(System.in);
        System.out.println("\n Creation of Tree");
        do
        {
            System.out.println(" Enter the element ");
            val=s.nextInt();
            obj.add(val);
            System.out.println("\n Do u want to enter more elements?");
            String str=s.next();
            ans=str.charAt(0);
        }while(ans=='y');
        System.out.println("The tree is... "+obj);
    }
}

```

Output

```

    Program to create Tree data structure
Creation of Tree
Enter the element
10
Do u want to enter more elements?
y
Enter the element
8
Do u want to enter more elements?
y

```

```

Enter the element
9
Do u want to enter more elements?
y
Enter the element
12
Do u want to enter more elements?
y
Enter the element
11
Do u want to enter more elements?
y
Enter the element
13
Do u want to enter more elements?
n
The tree is... [8, 9, 10, 11, 12, 13]

```

3.11 Map Interface

Map is a kind of data structure which associates the keys to the values. If there is a unique value present for each unique key then the mapping is called **one to one mapping**.

If for a unique key there are multiple values associated with it then it is called **many to one mapping**.

There is a difference between **Maps** and the **Sets**. The maps contain keys and values whereas the sets contain only values.

There are three classes which implement maps - **Hashtable**, **HashMap** and **TreeMap**.

3.11.1 Hashtable

The instance of **Hashtable** is created using which the methods **keys()** and **elements()** are invoked in order to get the keys and corresponding values. Then using **Enumerator** we can access every key and corresponding value of the **Map**.

Java Program[HashtableProg.java]

```

/*****
Program to display all the keys and corresponding values of the map
using Hashtable
*****/
import java.util.*;
public class HashtableProg
{

```

```
public static void main(String[] args)
{
    Hashtable h=new Hashtable();
    h.put("Accounts","Priyanka");
    h.put("Proof","Lekhana");
    h.put("Graphics","Nilesh");
    h.put("DTP","Archana");
    System.out.println("\n Displaying the Keys...");
    Enumeration E=h.keys();
    while(E.hasMoreElements())
        System.out.println(" "+E.nextElement());

    System.out.println("\n Displaying the values...");
    E=h.elements();
    while(E.hasMoreElements())
        System.out.println(" "+E.nextElement());
}
}
```

Output

Displaying the Keys...

Proof

DTP

Graphics

Accounts

Displaying the values...

Lekhana

Archana

Nilesh

Priyanka

3.11.2 HashMap

The instance of **HashMap** is created using which the methods **keySet()** and **values()** are invoked in order to get the keys and corresponding values. Then using iterator we can access every key and corresponding value of the **Map**.

Java Program[HashMapProg.java]

```
/******
Program to display all the keys and corresponding values of the map
using HashMap
*****/
import java.util.*;
```

```
public class HashMapProg
{
    public static void main(String[] args)
    {
        HashMap h=new HashMap();
        h.put("Accounts","Priyanka");
        h.put("Proof","Lekhana");
        h.put("Graphics","Nilesh");
        h.put("DTP","Archana");
        System.out.println("\n Displaying the Keys...");
        Collection c=h.keySet();
        Iterator i=c.iterator();
        while(i.hasNext())
            System.out.println(" "+i.next());
        System.out.println("\n Displaying the values...");
        c=h.values();
        i=c.iterator();
        while(i.hasNext())
            System.out.println(" "+i.next());
    }
}
```

Output

Displaying the Keys...

Accounts

Graphics

Proof

DTP

Displaying the values...

Priyanka

Nilesh

Lekhana

Archana

3.11.3 TreeMap

Displaying the map using TreeMap is similar to the HashMap. We use the **TreeMap** class instead of the **HashMap** class.

Java Program[TreeMap.java]

```
/******
Program to display all the keys and corresponding values of the map
using TreeMap
*****/
```

```
import java.util.*;
public class TreeMapProg
{
    public static void main(String[] args)
    {
        TreeMap t=new TreeMap();
        t.put("Accounts","Priyanka");
        t.put("Proof","Lekhana");
        t.put("Graphics","Nilesh");
        t.put("DTP","Archana");
        System.out.println("\n Displaying the Keys...");
        Collection c=t.keySet();
        Iterator i=c.iterator();
        while(i.hasNext())
            System.out.println(" "+i.next());
        System.out.println("\n Displaying the values...");
        c=t.values();
        i=c.iterator();
        while(i.hasNext())
            System.out.println(" "+i.next());
    }
}
```

Output

Displaying the Keys...

Accounts

Graphics

Proof

DTP

Displaying the values...

Priyanka

Nilesh

Lekhana

Archana

3.12 Multiple Choice Questions

Q.1 Pluggable look and feel and lightweight components are the features supported by ____.

☐ a swing

☐ b AWT

☐ c core Java

☐ d none of these